Laboratory – Neural Models: Learning, Depth, Activations, and Output Layers

Lab Exercise

August 7, 2026

Learning Objectives

By the end of this laboratory, you should be able to:

1. explain why a nonlinear hidden layer is needed for an XOR-like decision problem;
2. choose an output activation and loss that match a prediction task;
3. connect a PyTorch backward pass to the chain-rule/backpropagation ideas from the lecture;
4. test whether learning is actually occurring by inspecting losses, predictions, and gradients;
5. demonstrate the symmetry problem caused by identical weight initialisation;
6. use a Large Language Model (LLM) as an engineering assistant while retaining responsibility for problem specification, verification, and interpretation.

Background

A neural model is a parameterised function. For a layered network,

$$\boldsymbol{a}^{(\ell)} = W^{(\ell)}\boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}, \qquad \boldsymbol{h}^{(\ell)} = f^{(\ell)}\left(\boldsymbol{a}^{(\ell)}\right),$$

with $\boldsymbol{h}^{(0)} = \boldsymbol{x}$. Learning adjusts the parameters to reduce a loss. Backpropagation computes the derivatives required by the optimiser. For a binary output, a standard pairing is sigmoid plus binary cross-entropy. For one-of-K classification, a standard pairing is softmax plus cross-entropy, whose logit gradient is $\boldsymbol{p} - \boldsymbol{y}$.

A stack of affine layers without nonlinear hidden activations collapses into a single affine map. Consequently, nonlinear behaviour such as XOR cannot be represented by adding affine-only depth. Activation choice also influences optimisation: sigmoid can contribute small derivatives, while active ReLU units contribute derivative 1; deep products of Jacobian factors can therefore shrink or grow gradients.

AI Science vs. AI Engineering

AI Science asks what internal representation makes the desired behaviour possible and what learning signal causes that representation to emerge.

AI Engineering asks how to implement the model, generate code, choose numerical settings, test gradients, and detect failures. This laboratory deliberately requires both viewpoints.

Software Requirements

You need:

- Python 3;
- NumPy;
- PyTorch (CPU is sufficient; no GPU is required);
- a notebook environment or Python IDE;
- access to one freely available LLM assistant, for example ChatGPT, Gemini, Claude, or Microsoft Copilot.

Motivating Scenario: Redundant Safety Sensors

A small autonomous device has two redundant binary sensors, x_1 and x_2. The device should raise a *sensor-disagreement* warning exactly when one sensor is active and the other is inactive. The required decision rule is therefore XOR:

x_1	x_2	disagreement warning y
0	0	0
0	1	1
1	0	1
1	1	0

This is intentionally tiny: the objective is not to build a production safety system, but to make the representation and learning mechanism transparent enough to inspect.

Laboratory Workflow

Complete the tasks in order. Do not ask the LLM to "solve the lab" before you have written your own problem specification and model design.

Task 1: Understand the problem before coding

Goal: state precisely what the agent must compute and predict what a linear model can and cannot do.

1. Write the input space $\mathcal{X}$, output space $\mathcal{Y}$, and all four labelled examples.
2. Sketch the four points $(0,0), (0,1), (1,0), (1,1)$ in the x_1, x_2 plane and label them by class.
3. In one or two sentences, explain why one straight decision boundary cannot separate the two classes.
4. Predict what will happen if you train only a single affine transformation followed by a sigmoid output.

Checkpoint to record in your notebook: a short problem specification and your prediction about the linear model.

Think About It

A model can have many parameters and still have the wrong *kind* of representation. What scientific claim about representation does XOR let us test with only four data points?

Task 2: Design the intelligent agent

Before asking an LLM for code, specify a small neural model on paper.
Use the following baseline design:

$$2 \text{ inputs} \longrightarrow 2 \text{ hidden units} \longrightarrow 1 \text{ output}.$$

Choose:

- a nonlinear hidden activation (start with sigmoid or tanh; later try ReLU);
- a sigmoid output, because the target is one yes/no answer;
- binary cross-entropy as the loss;
- gradient-based optimisation.

In your own words, answer:

1. Why is the hidden nonlinearity scientifically necessary here?
2. Why is sigmoid plus binary cross-entropy a sensible engineering pairing for the output?
3. What evidence will count as successful learning? Define at least three checks (for example, final loss, all four predicted labels, gradient nonzero-ness, repeated-run behaviour).

Checkpoint to record: your model specification and validation criteria.

Think About It

The hidden units are not given target values. If the network learns XOR, what has determined what each hidden unit should compute? Relate your answer to the role of backpropagation.

Task 3: Use an LLM to generate a first implementation

Now use an LLM as an *engineering tool*. Ask it for concise PyTorch code that implements exactly the design you wrote in Task 2.
Your prompt should include all of the following constraints:

- the four XOR training examples explicitly;
- a 2–2–1 network;
- your chosen hidden activation;
- a sigmoid binary output (or, preferably in PyTorch, logits with `BCEWithLogitsLoss`);
- random weight initialisation;
- full-batch training for a few thousand lightweight CPU steps;

- printing the final loss and the four predictions;
- printing or otherwise exposing at least one gradient tensor after `backward()`.

A useful structure for your LLM request is:

> "Generate minimal PyTorch code for the following model and dataset. Do not change the architecture or task. After training, report the final loss, all four probabilities, thresholded labels, and one parameter-gradient tensor. Set a random seed for reproducibility and explain each test in one sentence."

Before running the code, inspect it. Identify where the forward pass occurs, where the scalar loss is formed, where reverse-mode AD is invoked, and where the optimiser changes parameters.
Checkpoint to record: the exact prompt you used, the generated code, and two changes (if any) that you made before execution.

Think About It

An LLM can produce syntactically correct code that implements the wrong experiment. Which parts of this laboratory could you verify from the code without running it, and which require execution and measurement?

Task 4: Execute, test, and diagnose the generated code

Run the implementation and collect evidence rather than trusting a single final number.
Part A – Basic learning check

1. Record the initial and final loss.
2. Record the four final probabilities and thresholded predictions.
3. Verify that all four labels are correct. If not, change only justified engineering settings such as learning rate, number of steps, seed, or optimiser; do not silently change the scientific task.

Part B – Backpropagation check After a forward pass and `backward()`, inspect the gradients of the first-layer weights. Explain what `parameter.grad` represents in terms of $\partial L/\partial W^{(1)}$.
If your code uses a mean loss over the four examples, state why the gradient is the average of the example-wise gradients.
Part C – Symmetry experiment Make a copy of the experiment in which *all weights are set to zero* before training (leave the architecture unchanged). Train again and inspect the two rows of the hidden-layer weight matrix over several steps.
Record whether they remain identical. Explain the result using the fact that identical hidden units compute the same output and receive the same gradient.
Part D – Activation experiment Train the random-initialised network three times, changing only the hidden activation: sigmoid, tanh, and ReLU. For each run, record:

- final loss;
- whether all four examples are classified correctly;
- the Euclidean norm of the first-layer gradient at an early training step.

Do not claim that one activation is universally "best" from four points. Instead, explain the observed differences for this experiment.
Checkpoint to record: a compact result table and one paragraph interpreting it.

Think About It

If a sigmoid unit is saturated, its derivative is close to zero. If a ReLU unit is negative, its derivative is zero. These are different mechanisms that can both produce a small gradient. How would you distinguish them by inspecting activations and pre-activations?

Task 5: Extend the task and reflect on the outcome

Convert the same sensor input into a three-class decision:

Class	Meaning
0	both sensors inactive: $(0, 0)$
1	sensors disagree: $(0, 1)$ or $(1, 0)$
2	both sensors active: $(1, 1)$

Keep the hidden representation small, but replace the single output with three logits and train using multiclass cross-entropy.
Ask your LLM to modify only the output/loss portion of the previous code. Before accepting the change, predict:

1. the shape of the final weight matrix;
2. the number of logits per example;
3. why softmax probabilities sum to one;
4. why the logit gradient has the form $\boldsymbol{p} - \boldsymbol{y}$.

Run the code and report the predicted class probabilities for all four inputs. For one example, print the softmax probability vector and verify numerically that its components sum to approximately 1.
Optional diagnostic: add the same constant (for example, 100) to all three logits before softmax and check that the probability vector is unchanged apart from floating-point roundoff. Then explain why stable implementations subtract the maximum logit before exponentiating.
Checkpoint to record: the modified output design, validation results, and your explanation of $\boldsymbol{p} - \boldsymbol{y}$.

Think About It

Next-token prediction in a language model can be viewed as classification over a very large vocabulary. Which parts of this tiny three-class experiment stay mathematically the same when the number of classes becomes tens of thousands, and which parts of the surrounding architecture change dramatically?

Suggested Result Table

Use a table like the following for the activation experiment.

Hidden activation	Final loss	4/4 correct?	Early $\|\nabla_{W^{(1)}} L\|_2$
Sigmoid			
Tanh			
ReLU			

Submission Instructions

Submit one notebook or Python file plus a short PDF/report containing:

1. your Task 1 problem specification and linear-separability explanation;
2. your model design and validation criteria from Task 2;
3. the exact LLM prompt(s) you used and a brief note describing any corrections you made to generated code;
4. the final code used for the binary XOR experiment and the three-class extension;
5. the requested loss, prediction, gradient, symmetry, and activation results;
6. answers to the reflection questions below.

Do not submit LLM output without verification. You are responsible for the correctness of the experiment, code, and interpretation.

Reflection Questions

Answer concisely but substantively.

1. What did the XOR experiment demonstrate about the difference between depth and nonlinearity?
2. In your successful run, what evidence showed that backpropagation supplied a useful learning signal rather than merely a nonzero gradient?
3. Why did identical/zero weight initialisation prevent the two hidden units from learning distinct features?
4. How did changing the hidden activation affect the gradient you observed? Distinguish the scientific explanation from the engineering observation.
5. Why must the output layer and loss be selected together according to the task?
6. Give one example where the LLM improved your engineering productivity and one example where human verification was essential.
7. Which tests in this laboratory would you keep if the model were scaled up, and which would become too expensive (for example, exhaustive finite-difference gradient checks)?

Responsible Use of LLMs

LLMs are useful for producing boilerplate code, explaining library calls, suggesting tests, and translating a mathematical design into a first implementation. They can also invent APIs, alter requirements, hide numerical mistakes, or provide an answer that appears plausible without being tested. Use an LLM as a collaborator for *AI engineering*, not as a substitute for the scientific reasoning that specifies what behaviour the model should exhibit and what evidence would support that claim.

Reference

The neural-model terminology and agent-oriented framing are consistent with standard undergraduate treatments such as S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. The laboratory itself follows the lecture's computational view of backpropagation, activation functions, task-dependent output layers, and softmax/cross-entropy learning.